USING MULTIPLE HANDLERS FOR A REQUIREMENT

As I mentioned before, requirements can have multiple handlers. This is usually the case when there are alternative ways of satisfying the requirement. Suppose users can view roles if they have the `Permission` claim with the value `View Roles` and have been with the business for at least six months, *or* they're in the `Admin` role. Now you have two alternative ways to authorize the user. You could add more code to the existing handler to check whether the user is in the specified role, but there would be a possibility of that code growing into a mess as more alternatives arise over time. Instead, you will implement an additional handler for your `ViewRolesRequirement`. This time, however, you will take an alternative approach to crafting the handler that types it specifically to the requirement and, therefore, negates the need to filter all pending requirements.

You will derive from the abstract `AuthorizationHandler<TRequirement>` class, where `TRequirement` represents the requirement type the handler is designed for. The handler logic is placed in an overridden `HandleRequirementAsync` method which, in addition to an `AuthorizationHandlerContext`, takes the requirement type as a parameter. The following listing shows the claims check refactored into an `AuthorizationHandler<TRequirement>` class named `HasClaimHandler`, which is placed in a folder named AuthorizationHandlers.

Listing 10.23 Handling authorization requirements